Multi-Agent Systems and Agentic AI Practical Examination

Rowan d'Auria
Team members: Basia Koch, Funmi Looi-Somoye

June 24, 2026

GitLab repository	`a8_coursework/rd761`
Main training logs	`felsomoye-university-of-cambridge/tunix`

Contents

Part I: GRPO Finetuning of Gemma 3

I.1 Reproducing the Baseline

Run identity. Commit `7e696c42` (tag `run/k2-baseline`) on branch `baseline-fls`, W&B run `jgs4c6kl`, wall-clock 4h 41m, completing 3364 GRPO steps (full schedule 3738×0.9).

Baseline patches. The baseline did not run as shipped because TFDS/GSM8K loading failed with a protobuf `FieldDescriptor.label` error, caused by a `protobuf` 7.x and `tensorflow-datasets` 4.9.9 compatibility issue. The fix was to pin `protobuf==6.31.1` in `requirements.txt`; this changed only dependency plumbing, not the model, reward functions, data split, or training hyperparameters.

Accuracy. On the GSM8K test split (seed-42 shuffle, first 64 examples, greedy decoding) the base `gemma-3-1b-it` scores $33/64 = 51.56\%$ exact, while the baseline LoRA GRPO checkpoint (step 3364) scores only $2/64 = 3.12\%$, substantially worse than base, indicating a training failure rather than a checkpoint-restore bug.

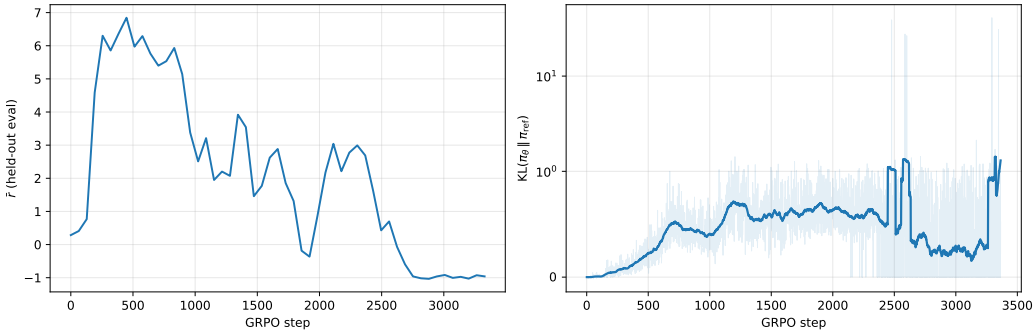

Figure 1: Baseline GRPO training vs. step (W&B run `jgs4c6kl`). Left: held-out eval reward $\bar{r}$ (mean per-rollout total reward summed over the four reward terms, every 64 steps). Right: per-step reference-KL $\mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})$ (faint raw trace, bold 64-step moving average; symlog y-axis, as KL sits near 0.5 but spikes to $\sim$40 during the late collapse).

Training curves. Eval reward peaks around 6.8 near step 460, then declines through an oscillating descent to about -1 by step $\sim$2750 and flatlines; KL holds near 0.5 with spikes around steps 2500 and 3300. The policy learned reward-correlated behaviour early but drifted from the reference without preserving GSM8K accuracy.

I.2 Understanding the Baseline

Policies (a). All three share the frozen Gemma-3-1B weights and differ only in LoRA adapters. π_θ is the actor (LoRA matrices in `scripts/model.py`, the only trainable parameters); π_{ref} is the frozen base (zero-adapter) policy passed to `RLCluster`, the KL anchor with no trainable parameters; π_{old} is the actor snapshot that generated the rollouts. With `NUM_ITERATIONS=1` Tunix skips a separate old-actor pass: inside the loss the old log-probabilities are the current per-token log-probabilities under `stop_gradient`.

Group size and estimator (b). $K = 2$ (`NUM_GENERATIONS=2` in `scripts/config.py`). With the default `advantage_estimator="grpo"` Tunix uses the standard group-normalised $\hat{A}_i = (r_i - \bar{r}_q)/(s_q + 10^{-6})$, with $\bar{r}_q, s_q$ the mean and sample s.d. of the K rewards for a prompt, the ordinary group-mean baseline, not the leave-one-out (`rloo`) estimator.

Reward decomposition (c). `scripts/rewards.py` sums four terms. `match_format_exactly`/`match_format_approximately` are format-shaping (they reward the `<reasoning>`/`<answer>` tags regardless of the arithmetic); `check_answer` is the true correctness reward (largest payoff for the exact GSM8K number, partial/negative otherwise); `check_numbers` is a numeric fallback. With correctness alone, early on most siblings score the same (usually zero), so the within-group $s_q \to 0$ and the standardised advantages vanish; the shaping terms make two siblings differ before the model can do the maths.

Clipped surrogate (d). Implemented as `grpo_loss_fn` in `tunix/rl/algo_core.py` (selected by `policy_loss_fn="grpo"`): it forms the new/old per-token log-prob ratio ρ and minimises $\max\{-\hat{A}\,\rho,\ -\hat{A}\,\mathrm{clip}(\rho, 1-\varepsilon, 1+\varepsilon)\}$ (a loss, hence the sign flip vs the lecture objective). `EPSILON=0.2` clips ρ to $[0.8, 1.2]$. The deviation from the lecture form is granularity: ratios are per completion-token, aggregated with `loss_agg_mode="sequence-mean-token-mean"` and the scalar group advantage broadcast over tokens, rather than a single sequence-level ratio.

I.3 Improving on the Baseline

Motivation. Modification 1 (group size). The baseline uses $K = 2$ with the standardised advantage $\hat{A}_i = (r_i - \bar{r})/\sigma_r$ (see I.2). With two samples σ_r is poorly estimated, and a group is *degenerate* whenever both completions earn equal reward ($\sigma_r = 0$, advantage ill-defined, no gradient); even non-degenerate groups give a tiny effective sample size. Raising K above 2 is the most direct theory-motivated fix: it should reduce degenerate groups, condition σ_r, and lower advantage variance. A team sweep of $K \in \{2, 4, 8, 16\}$ (Table 1), alongside further single-knob parameter runs (Table 2), settled the operating point at $K = 8$, which gave the clearest eval gain and the most stable training.
Modification 2 (reward). Once $K = 8$ trains stably it exposes a *reward-hacking* failure: with format and correctness weighted roughly equally, the policy formats almost perfectly while answering only ≈half correctly. We downweight the format terms and replace the coarse step-function correctness term with a denser, accuracy-weighted clipped-exponential reward, then resume the $K = 8$ checkpoint for a further ∼2.5k steps (5864 total). See Appendix A.4 for the pseudocode implementation.

Controlled-comparison design. The comparison holds the GSM8K test split, greedy decoding, and seed-42 ordering fixed, and evaluates the full 1319-example test set for the base model, the $K = 2$ baseline, the matched $K = 8$ run, and the $K = 8$ new-reward run. All three GRPO runs use the same data and the same optimisation schedule (nominal 6516 steps, $\times 0.9 = 5864$ training steps). We normalise by training step, not wall-clock or FLOPs: this fixes the gradient updates, schedule, and data ordering, so any difference traces to the per-step advantage estimate. The $K = 2$ baseline is the I.1 configuration trained to this same schedule. Uncertainty is a non-parametric bootstrap over the 1319 prompts (10,000 resamples, seed 42; per-question results in `evaluation/`), *paired* for cross-run differences.

Training dynamics and diagnostic. Figure 2 overlays the held-out curves (every 64 steps). Reward is comparable only *within* a fixed reward function, so the left panel shows the two default-reward runs and the new-reward run is judged by KL and accuracy. The $K = 8$ reward is sustained around 7.5, whereas $K = 2$ peaks early (step ∼300) and collapses by step ∼1000 to the −2.5 format-penalty floor. KL (right panel) tells the same story: $K = 2$ spikes at the collapse (raw KL ≈ 10, 64-step mean ≈ 1) and stays erratic, while both $K = 8$ runs hold a bounded ≈ 0.2–0.3. The diagnostic (Figure 3), mean eval completion length (a known GRPO failure mode) pins the mechanism: at step ∼1000 $K = 2$ collapses to near-empty completions, with an unstable blow-up to ∼600 tokens over steps 2300–3100 before settling at a ∼40-token mode; both $K = 8$ runs hold a stable ∼300–360 tokens (new-reward ≈ 362 vs default ≈ 352).

Why response length, not advantage variance. The suggested advantage-variance diagnostic is uninformative here: $\hat{A}_i = (r_i - \bar{r})/\sigma_r$ is standardised *per group* (Tunix sets $\hat{A}_i = 0$ when $\sigma_r = 0$), so $\mathrm{Var}(\hat{A}_i) \approx 1$ by construction. The genuinely informative quantity, the share of degenerate groups tied

to K_{eff} (I.4), was not logged, so we use mean completion length, the strongest logged GRPO-failure-mode signal.

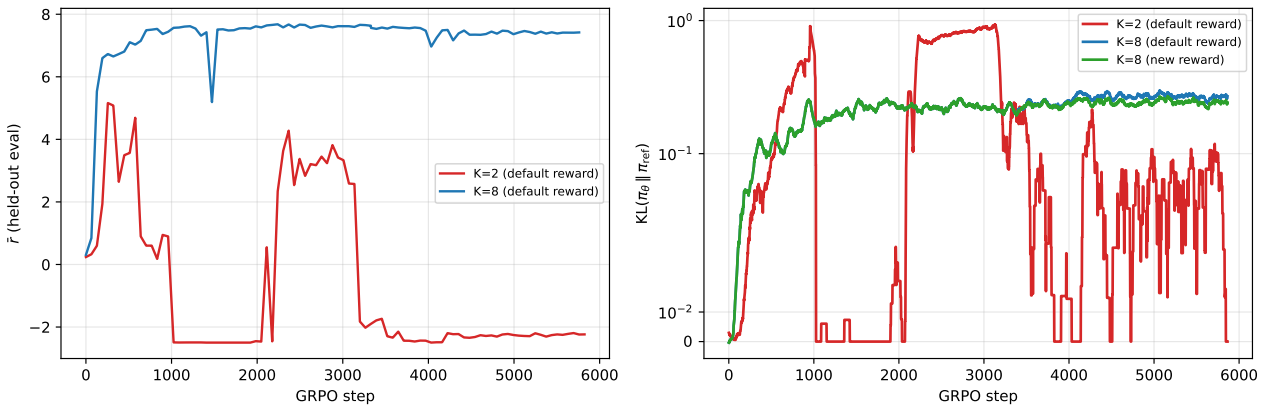

Figure 2: Training dynamics (held-out, every 64 steps). Left: eval reward $\bar{r}$ for the two default-reward runs (reward is comparable only within a fixed reward function). Right: reference-KL $\mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})$ for all three runs (64-step moving average, symlog y-axis).

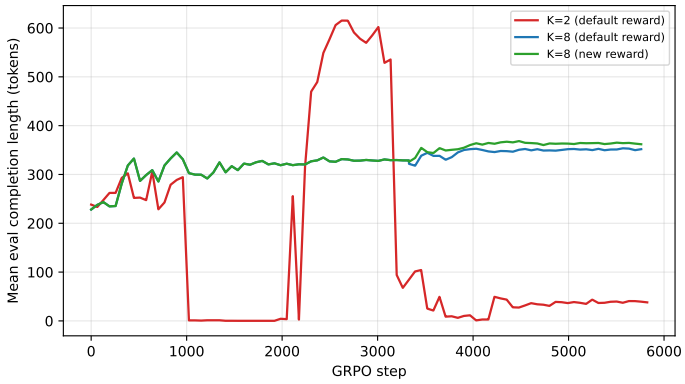

Figure 3: GRPO-failure-mode diagnostic: mean held-out eval completion length (tokens) over training.

Findings and theory. Exact-match answer accuracy on the 1319-example GSM8K test set (greedy decoding): the $K = 2$ baseline collapses to 0.08%, below the base model's 47.38%; $K = 8$ recovers to 56.03% (+8.64 over base) and the $K = 8$ correctness-heavy reward reaches 58.53% (+11.14 over base). Per-run seeds, steps, and 95% CIs are in Table 3; all six paired-difference CIs are in Table 4 (Appendix A.3).

The reward-hacking signature is clear: format pass-rate holds across the two $K = 8$ runs (94.8% $\rightarrow$ 94.3%, difference not separable from 0) while answer accuracy rises. Yet contrary to Modification 2's intent the reward change under-delivered: its +2.50 gain over the matched $K = 8$ run is not separable from zero, so the binding constraint is not reward format. All checkpoints still answer in the training format on unrelated prompts (Appendix A.5). The $K = 2$ collapse is the within-group-variance pathology from the lectures: with σ_r often zero or noisy, the standardised advantage carries little signal and the policy drifts (the KL spike). Raising K conditions σ_r and raises K_{eff}, lowering gradient-estimator variance and keeping updates inside the KL budget.

Limitations. Each setting is a single seed, and both $K = 8$ runs share phase 1 (3364 default-reward steps) stitched onto each run's $\sim$2.5k-step phase 2; a clean reward claim needs a second seed.

I.4 GRPO Theory

Question 1: Group-Normalised Advantages

(i) Since $\mathbb{E}[r_i - \bar{r}] = 0$ and h_i is fixed,

$$\mathbb{E}[X_i] = \frac{h_i}{\sigma}\mathbb{E}[r_i - \bar{r}] = 0, \qquad \mathbb{E}[\hat{g}] = \frac{1}{K}\sum_i \mathbb{E}[X_i] = 0. \tag{1}$$

This conditions on fixed actions and i.i.d. rewards. In the actual policy gradient, reward depends on the sampled action, so score and reward may be correlated.

(ii) The shared random term is $-h_i\bar{r}/\sigma$ in $X_i = h_i(r_i - \bar{r})/\sigma$. For $i \neq j$,

$$\mathrm{Cov}(r_i - \bar{r}, r_j - \bar{r}) = 0 - \frac{\sigma^2}{K} - \frac{\sigma^2}{K} + \frac{\sigma^2}{K} = -\frac{\sigma^2}{K}, \tag{2}$$

$$\therefore \quad \mathrm{Cov}(X_i, X_j) = -\frac{1}{K}h_i h_j^\top. \tag{3}$$

The coefficient is negative: increasing r_i raises the shared mean and lowers every other centred reward. (For unequal h_i, h_j, the outer-product matrix itself has no semidefinite sign.)

(iii) Since $\mathrm{Var}(X_i) = (K-1)h_i h_i^\top/K$, Eq. (3) gives, with $\bar{h} = K^{-1}\sum_i h_i$,

$$\mathrm{Var}(\hat{g}) = \frac{1}{K^2}\left[\frac{K-1}{K}\sum_i h_i h_i^\top - \frac{1}{K}\sum_{i\neq j} h_i h_j^\top\right] \tag{4}$$

$$= \frac{1}{K^2}\sum_i (h_i - \bar{h})(h_i - \bar{h})^\top. \tag{5}$$

The naive i.i.d. result would be $\mathrm{Var}(X_1)/K = (K-1)h_1 h_1^\top/K^2$ and omits the cross-covariances. Because covariances are matrices, effective sample size is direction-dependent. For a direction u, define

$$K_{\text{eff}}(u) := \frac{\mathrm{Var}(u^\top X_1)}{\mathrm{Var}(u^\top \hat{g})} = \frac{K(K-1)(u^\top h_1)^2}{\sum_i \{u^\top (h_i - \bar{h})\}^2}. \tag{6}$$

For bounded, non-degenerate score dispersion this is $O(K)$. If $h_i = s_i v$ are aligned, only the variation of the s_i remains; if all h_i are identical then $\sum_i \hat{A}_i = 0$, so $\hat{g} = 0$ and K_{eff} is infinite.

Question 2: KL-Regularised Policy Updates and Clipping

(i) Using a multiplier for $\sum_a \pi(a) = 1$, stationarity gives $A_t(a) - \eta^{-1}(\log(\pi(a)/\pi_t(a)) + 1) + \lambda = 0$. Hence

$$\boxed{\pi_{t+1}(a) = \frac{\pi_t(a)e^{\eta A_t(a)}}{\sum_b \pi_t(b)e^{\eta A_t(b)}}.} \tag{7}$$

Strict concavity gives uniqueness on the support of π_t. Small η keeps the update close to π_t; large η concentrates more strongly on high-advantage actions.

(ii) For the exact advantage, $J(\pi_t) = \mathbb{E}_{\pi_t}[A^{\pi_t}] = 0$. Since π_t is feasible, optimality implies

$$J(\pi_{t+1}) - \frac{1}{\eta}\,\mathrm{KL}(\pi_{t+1}\|\pi_t) \geq 0, \qquad \therefore \qquad J(\pi_{t+1}) \geq 0 = J(\pi_t). \tag{8}$$

Neural GRPO instead uses a noisy finite-group advantage and approximate SGD optimisation, so the exact-advantage/optimiser premise, and therefore the guarantee, can fail.

(iii) For $\hat{A}_t > 0$, clipping removes the incentive to raise the sampled ratio above $1+\varepsilon$; for $\hat{A}_t < 0$, it removes the incentive to lower it below $1-\varepsilon$. This is a samplewise, one-sided soft trust region, not a hard ratio constraint. A joint KL constraint instead bounds average change over the full policy, allowing larger changes on rare actions; clipping ignores unsampled actions and does not guarantee any particular KL bound.

Question 3: Mixture Proposals and Importance Correction

Write $p = \pi_{\text{old}}(\cdot \mid q)$, $m = \pi_{\text{mix}}(\cdot \mid q)$, and $u = \pi_{\text{unif}}(\cdot \mid q)$.

(i) Since $\mathbb{E}_p[f] = \mathbb{E}_m[wf]$, the importance-corrected estimator is

$$\hat{g}_{\text{mix}} = \frac{1}{K}\sum_i w_i \nabla_\theta \log \pi_\theta(a_i \mid q)\hat{A}_i, \qquad \boxed{w_i = \frac{p(a_i)}{m(a_i)} = \frac{p(a_i)}{(1-\alpha)p(a_i) + \alpha u(a_i)}.} \tag{9}$$

The support condition holds because $m(a) \geq (1-\alpha)p(a)$ for $\alpha < 1$.

(ii) As $\alpha \to 0$, $m(a_i) \to p(a_i)$ and $w_i \to 1$. With the same actions and group advantages, $\hat{g}_{\text{mix}} \to \hat{g}_{\text{GRPO}}$.

(iii.a)

$$V^m(q) = (1-\alpha)V^p(q) + \alpha V^u(q). \tag{10}$$

(iii.b) The weights correct the sampling distribution but do not make the unweighted mixture mean estimate V^p. The target-centred term is

$$w_i \frac{r_i - V^p(q)}{\sigma_p}, \qquad V^p(q) = \mathbb{E}_m[w(a)R(q,a)]. \tag{11}$$

For an unbiased finite-sample baseline, use the independent leave-one-out estimate

$$b_{-i} = \frac{1}{K-1}\sum_{j \neq i} w_j r_j, \qquad \boxed{\hat{g}_{\text{mix}} = \frac{1}{K}\sum_i w_i \nabla \log p(a_i \mid q)\frac{r_i - b_{-i}}{\sigma_p}.} \tag{12}$$

Here $\mathbb{E}[b_{-i}] = V^p$, b_{-i} is independent of a_i, and $\mathbb{E}_m[w\nabla \log p] = 0$. This establishes unbiasedness when σ_p is treated as fixed; empirical standardisation introduces ratio-estimator bias.

(iv) There is no universal variance ordering. Before weighting,

$$\text{Var}_m(R) = (1-\alpha)\,\text{Var}_p(R) + \alpha\,\text{Var}_u(R) + \alpha(1-\alpha)(V^p - V^u)^2. \tag{13}$$

Thus mixing increases advantage variance if, for example, $\text{Var}_u(R) \geq \text{Var}_p(R)$ and $V^u \neq V^p$. It decreases it when $V^u = V^p$ and $\text{Var}_u(R) < \text{Var}_p(R)$. For the importance-weighted gradient integrand $F = \nabla \log p(R - V^p)/\sigma_p$, $\mathbb{E}_m\|wF\|^2 = \mathbb{E}_p[w\|F\|^2]$; variance rises when large residuals occur mainly where $w > 1$, but can fall if the mixture oversamples them, making $w < 1$.

(v.a) For $0 < c < 1$, $w_i = c^T = \exp(T \log c) \to 0$ exponentially. Uniform mixing adds mass to tokens that were unlikely under the old policy, so on such samples the old-to-mixture ratio is typically below one.

(v.b) Almost all long-response weights then vanish, leaving a very small effective sample size and an estimate dominated by rare non-negligible weights. Clipping log-weights prevents this exponential dynamic range and reduces variance, at the cost of bias because the change of measure is no longer exact.

Part II: Adaptive Planning in `cmbagent_lg`

Studied branch/commit: `adaptive-planning` at `commit d7d0592`.

II.1 Workflow Diagram

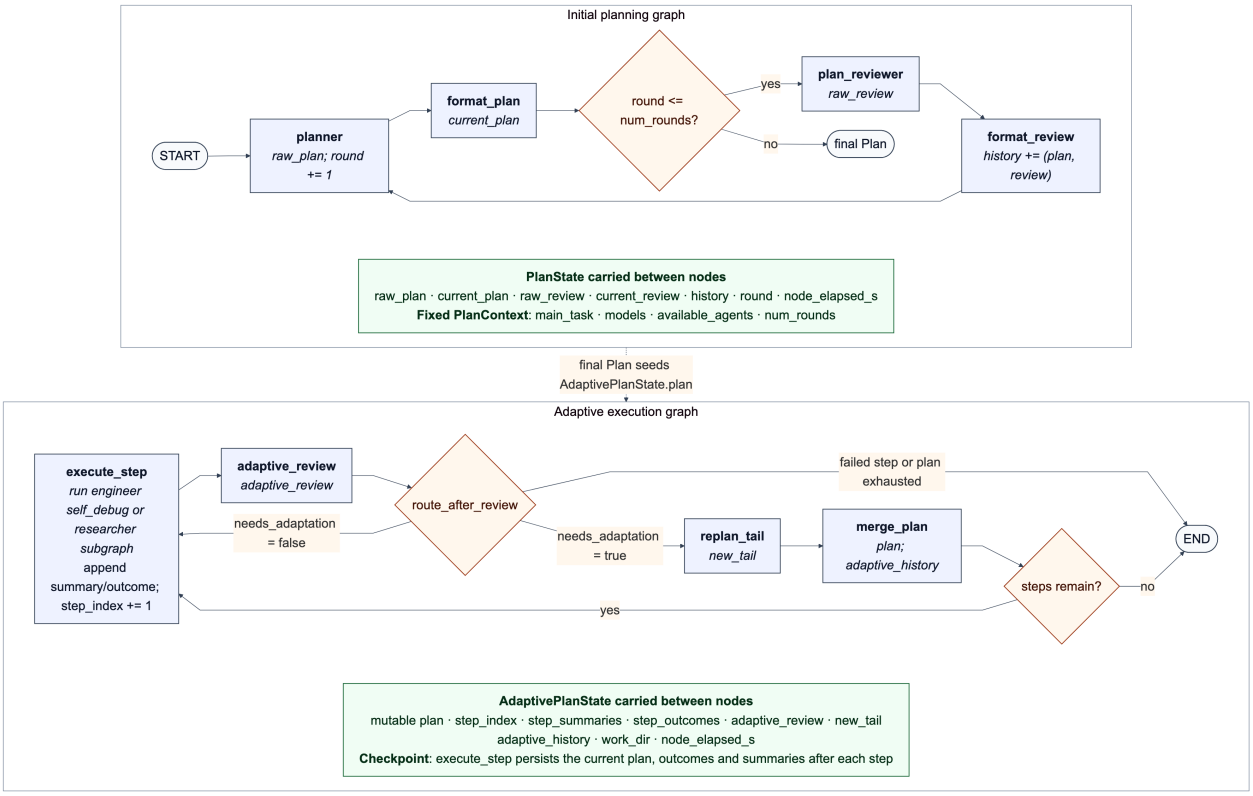

Figure 4: Manually extended Mermaid rendering of the implemented workflow at commit `d7d0592`. The generated topology is supplemented with routing conditions, state fields, node updates and the caller-mediated hand-off between the separately compiled graphs.

Walkthrough. The initial graph uses a bounded propose–critique loop. It performs `num_rounds` reviews and one final planner pass, then a caller places `current_plan` into the separately compiled adaptive graph. There, `execute_step` reuses `deep_research.run_step`: it dispatches the current step to the engineer's bounded self-debug graph or the researcher graph, records a summary and structured outcome, checkpoints the run, and increments `step_index`.

The adaptive reviewer is called after every executed step. Given the main task, latest summary and outcome, and current unexecuted tail, its structured `needs_adaptation` field triggers `replan_tail`. The replanner may replace every not-yet-executed step, including their number, contents and assigned agents. `merge_plan` holds the completed prefix `plan[:step_index-1]` and its outputs fixed, then appends the replacement tail. Otherwise execution advances directly. Routing stops after a failed step or when `step_index > len(plan)`. This guarantees termination only while the remaining plan is finite and is not repeatedly extended. Unlike the initial planning and inner self-debug loops, the adaptive loop has no total-step or revision bound, so the implementation does not provide an unconditional termination guarantee.

II.2 Problems and Failure Modes

1. Failure-blind and unbounded routing. The outer controller always invokes the adaptive reviewer, even after the final or a failed step, but `route_after_review` then terminates on failure before consulting `needs_adaptation`. It therefore cannot decompose, replace or reassign precisely the step for which feedback is most valuable. In the other direction, a successful step may trigger a replacement tail of arbitrary length. `maximum_number_of_steps_in_plan` appears only in initial-planning prompts and is neither a schema constraint nor an outer execution budget. Repeated extensions can prevent completion and produce unbounded cost. This contradicts the study guide's central control principle: feedback loops need an orchestration-enforced attempt budget, rather than relying on an LLM to stop itself.

2. A lossy, uncalibrated feedback signal. The adaptive reviewer observes only the latest text summary, a small outcome dictionary and the remaining steps. It does not receive the full completed history, execution verdicts, workspace manifest or artefact contents that are available elsewhere. Consequently its closed-loop observation is a lossy proxy for the environment: a summary may omit an invalid assumption or corrupt file, causing a false negative, while a stylistic surprise may cause an unnecessary rewrite. A single stochastic critic supplies both the binary trigger and its justification, with no confidence, deterministic invariant or independent check. This is weaker than the study guide's two-gate distinction between "did it run?" and "did it achieve the goal?", and makes control vulnerable to shared model blind spots.

3. Unverified and irreversible revisions. The initial plan receives fixed-round critique, whereas an adaptive tail is generated once and spliced into execution without review. Pydantic guarantees field types and permitted agent names, but not task coverage, dependency order, artefact availability or a maximum length. Moreover, the completed prefix is always immutable. If new evidence invalidates an earlier result, the controller can only build on or work around it, not roll back the affected step and its dependants. Finally, `run_step` checkpoints before the subsequent merge; the revised plan and `adaptive_history` are not persisted by the merge. A crash can therefore restore the pre-revision plan, undermining the audit trail and reproducibility expected from an observable closed-loop system.

II.3 Proposed Improvements

1. Add a budgeted status and recovery router. Route immediately after `execute_step`: end without another LLM call when the task is complete; review successful non-final steps; and send failures to a `recover_failure` node. That node should replan from the failed step, so the fixed prefix ends at `step_index-2`, using its execution and goal verdicts as mode-aware feedback. Add `total_steps` and `adaptation_count` to state, with context limits `max_total_steps` and `max_adaptations`; reject a tail longer than the remaining budget and return an explicit exhausted verdict. This both uses failure

feedback and guarantees termination. The trade-off is more routing logic and the risk of stopping a task whose useful recovery needed a larger budget.

2. Review a typed, evidence-rich observation. Introduce an `AdaptiveObservation` containing all prior outcomes, the two latest verdicts, accumulated summaries, workspace manifest, and compact artefact checks such as file existence, schema and hashes. Use deterministic rules for hard signals, including failure, missing dependencies and violated invariants; reserve the LLM reviewer for ambiguous semantic changes. Require confidence and evidence fields, and invoke a different critic or a second vote only below a confidence threshold. This improves observability and separates system checks from model judgement. It costs tokens and latency, while excessive raw context could itself distract the reviewer, so artefacts should be summarised rather than inserted wholesale.

3. Validate, persist and selectively roll back revisions. Insert `validate_tail` before `merge_plan`. It should enforce the remaining step budget and agent constraints deterministically, then ask a plan critic to check goal coverage, dependencies and consistency with completed artefacts. A rejected candidate returns to `replan_tail` within the same adaptation budget. Extend the review schema with an optional `restart_from` index; when evidence invalidates prior work, archive the affected artefacts and re-execute that suffix. After acceptance, atomically checkpoint the old and new tails, reason, evidence and revision ID. This restores verification, controlled rollback and a failure audit trail, at the cost of extra critic calls, storage, and a new risk that an incorrect rollback discards valid work.

A Appendix: Extra Experimental Details

A.1 Group-size (K) sweep

We swept $K \in \{2, 4, 8, 16\}$ on a 64-example GSM8K subset (greedy decoding, seed 42). The $K = 16$ checkpoint is from approximately 2500 steps; all others use 3364 steps.

K	W&B run	Commit	Wall time	Steps	Accuracy	Partial	Format
2	jgs4c6kl	bd98193	4.69 h (4 h 41 m 36 s)	3364	3.12% (2/64)	6.25%	12.50%
4	x4j7yhdp	1db3d25	6.63 h (23,867 s)	3364	54.69% (35/64)	59.38%	89.06%
8	m3xp6k97	b66b376	7.41 h (26,670 s)	3364	59.38% (38/64)	62.50%	95.31%
16	xqbl406c	17c90d5	8.03 h (28,923 s)	≈2500 (run failed @2991)	56.25% (36/64)	59.38%	92.19%

Table 1: Group-size sweep on the 64-example GSM8K eval subset (greedy decoding, seed 42). For each run the exact commit, wall-clock time, and GRPO steps completed are reported (cf. I.1). Accuracy is exact-match; *partial* additionally credits a correct final number with imperfect formatting; *format* is the format-check pass rate.

Performance rises sharply from $K = 2$ to $K = 4$, then plateaus. $K = 8$ matches $K = 16$ at lower cost and was used for the flagship runs.

A.2 Other training-parameter changes

We also varied the learning rate, LoRA rank/α, KL coefficient β, data, and completion penalties. Runs use seed 42 and the 64-example greedy evaluation unless noted.
$K = 8$ variants have stable eval reward, but accuracy varies. None exceeds the 59.38% $K = 8$ baseline. Results are single-seed and indicative only.

A.3 Full-test controlled comparison

Full breakdown of the I.3 controlled comparison (referenced from the headline results in I.3).

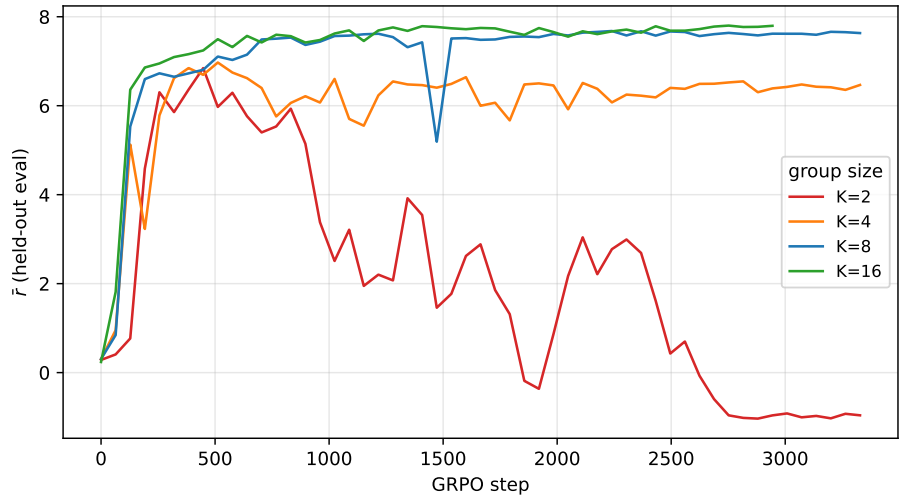

Figure 5: Held-out eval reward by group size (raw values, logged every 64 steps).

Run	Change vs baseline	K	Wall	W&B	LoRA acc (64)
LR 1e-5	LR 3e−6 → 1e−5	8	8h03	`hozux9t6`	29.69% (19/64)
LoRA rank 128	rank/α 64 → 128 (with K=8)	8	8h32	`aoz8dtkp`	29.69% (19/64)
LoRA rank 128	rank/α 64 → 128	2	5h29	`v5cvlwkm`	32.81% (21/64)
β=1e-6	β 0.08 → 10^{-6} (KL ≈ off)	2	3h26	`8rmv0hgg`	51.56% (33/64)
β=0.32	β 0.08 → 0.32 (4× leash)	2	1h07	`oet2tfjd`	0.00% (0/64)
Length penalty	+ length penalty	2	2h34	`jcp0b5cy`	4.69% (3/64)
Length penalty	+ length penalty (batch 2 → 1)	8	5h17	`cyay16mj`	50.00% (32/64)
Hard/medium data	hard/medium curriculum data (with K=8)	8	7h36	`6yiowy1y`	46.88% (30/64)
Empty penalty	+ empty-completion penalty	2	6h17	`dsi65u1z`	14.06% (9/64)

Table 2: Further single-variable runs (all seed 42, 3364 steps, greedy 64-example GSM8K eval). The rank-128 K=8 run confounds rank with group size. The empty-penalty run has no `runs/` folder; its accuracy is from the step-3364 `eval_dumps`. The batch-size change in the last length-penalty run, and the K=8 in the hard/medium-data run, each confound the change with group size; the hard/medium-data run's first attempt (W&B `hlo1w4go`) also crashed at step 832 on a VM reboot before this clean rerun.

Run	Modification	Seed	Steps	Accuracy	95% bootstrap CI
Base `gemma-3-1b-it`	no fine-tuning	42	–	625/1319 = 47.38%	[44.81%, 50.11%]
K=2 LoRA	default reward (baseline config)	42	5864	1/1319 = 0.08%	[0.00%, 0.23%]
K=8 LoRA	group size K: 2 → 8, default reward	42	5864	739/1319 = 56.03%	[53.45%, 58.76%]
K=8 new reward	K = 8 plus correctness-heavy reward	42	5864	772/1319 = 58.53%	[55.88%, 61.18%]

Table 3: Full-test GSM8K accuracy (1319 prompts, greedy decoding, seed-42 split) with non-parametric bootstrap 95% CIs (10,000 resamples, seed 42).

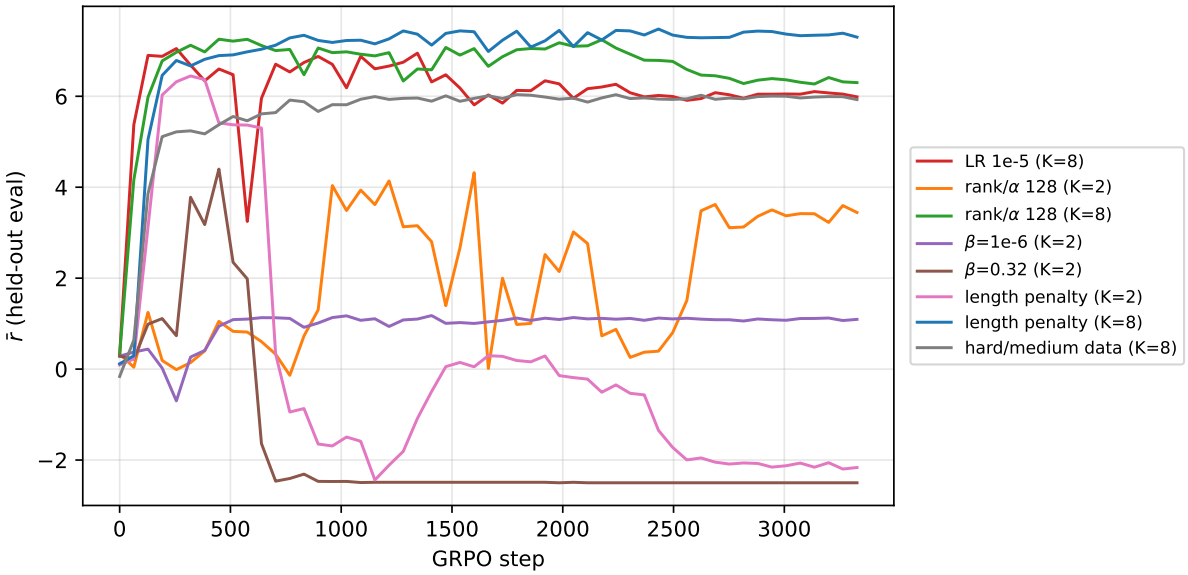

Figure 6: Held-out eval reward for logged runs (raw values, every 64 steps).

Comparison ($B - A$)	Δ accuracy	95% CI
K=2 − base	−47.31%	[−50.04%, −44.66%]
K=8 − base	+8.64%	[+5.84%, +11.45%]
K=8+rw − base	+11.14%	[+8.42%, +13.80%]
K=8 − K=2	+55.95%	[+53.37%, +58.68%]
K=8+rw − K=2	+58.45%	[+55.80%, +61.11%]
K=8+rw − K=8	+2.50%	[−0.08%, +5.00%]

Table 4: Paired bootstrap differences in exact-match accuracy (same 1319 prompts, 10,000 paired resamples, seed 42). $\Delta = \text{acc}(B) - \text{acc}(A)$; a 95% CI excluding 0 indicates the pair differs at the 5% level. Every comparison is significant except K=8+rw − K=8, whose CI includes 0.

A.4 Reward reweighting: pseudocode for the changes vs the baseline

Pseudocode for the parts of the GRPO reward that differ between the baseline (`scripts/baseline_scripts/rewards.py`) and the I.3 new-reward run (`scripts/rewards.py`, branch `reward-reweight`, commit `e4f6d69`). The four reward functions are still summed then GRPO-normalised within the group, and correctness is still gated on the template parsing; only the correctness *shape* and the term *magnitudes* change, shifting the budget from format ($5.5 \to \approx 1.0$) to correctness ($4.5 \to 5.5$) to counter format-vs-correctness reward hacking.

```
# check_answer: coarse ladder (incl. a negative penalty)  ->  smooth graded curve
# BASELINE:
  if   guess == true:                 score = 3.0
  elif guess.strip() == true.strip(): score = 1.5
  else: ratio = guess / true
        score = 0.5 if 0.9<=ratio<=1.1 else 0.25 if 0.8<=ratio<=1.2 else -1.0
        on parse error: score = -0.5
# NEW: graded_correctness(guess, true, MAX=CORRECT_MAX=5.0), shared by both answer terms
  if guess.strip() == true.strip(): return MAX
  err = |guess - true| / max(|true|, 1e-6)        # relative error; parse error -> 0
  return MAX * exp(-CORRECT_K * err)              # CORRECT_K=3.0; smooth, in [0, MAX]

# match_format_exactly:  3.0            ->  FORMAT_EXACT = 0.5   (entry fee)
# match_format_approx.:  +/-0.5 per tag ->  +/-FORMAT_TAG = +/-0.1 per tag (5 tags)
# check_numbers:  1.5 if guess==true else 0  ->  graded_correctness(.., MAX=0.5)
```

A.5 Qualitative responses to general prompts

To probe behaviour outside GSM8K, we asked each checkpoint the same simple factual questions. The following transcript excerpts show one failure from each trained model.

Step 3364, default reward: Mexico

```
> What is the capital of Mexico?
<reasoning>
The problem asks for the capital of Mexico. Mexico is a country in North
America. The capital city of Mexico is Mexico City. We need to find the
capital of Mexico.
</reasoning>

<answer>1</answer>
```

Step 5864, default reward: Italy

```
> What is the capital of Italy?
<reasoning>
The problem asks for the capital of Italy. Italy is a country in Europe. The
capital of Italy is Rome. Rome is a major city in Italy. It is the seat of
the Italian government and is a significant historical and cultural center.
</reasoning>

<answer>1</answer>
```

Step 5864, new reward: France

```
> What is the capital of France?
<reasoning>
The question asks for the capital of France. The capital of France is Paris.
We need to find the capital of France.
</reasoning>

<answer>10</answer>
```

All three models adopt the GSM8K response format, but each can replace a fact stated correctly in its reasoning with a numerical final answer. These hand-selected examples indicate a brittle output policy rather than a reliable loss of factual knowledge.

B AI Assistance Declaration

I used generative AI tools, including OpenAI Codex, while completing this coursework. These tools assisted with code development and debugging, the interpretation of error messages, and revisions to the report's wording and LaTeX presentation. I reviewed and, where necessary, modified all generated material before including it. I take full responsibility for the submitted work and can explain the code, analysis and arguments it contains.